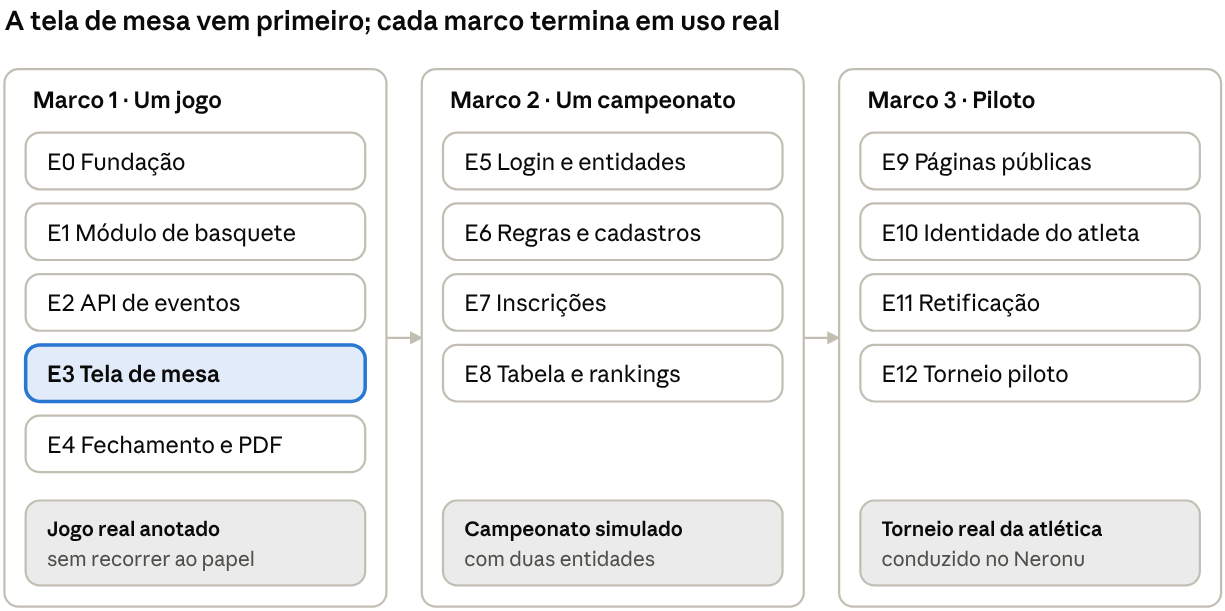
Neronu — Plano de implementação

Nov 10, 2026 · @Adama

Este plano transforma o SDD do Neronu em 13 etapas de trabalho (E0 a E12), na ordem em que devem ser feitas por uma pessoa, sem datas fixas.

1. Visão geral

O trabalho segue os três marcos do SDD, e cada marco só começa quando o critério do anterior foi atingido em uso real.



roteiro do Neronu · 13 etapas em 3 marcos, com o critério de cada marco

As etapas são feitas em sequência, de cima para baixo e da esquerda para a direita. A caixa cinza no pé de cada marco é o critério que o encerra.

Marco	Etapas	Entrega
1. Um jogo	E0 a E4	Partida anotada do início ao PDF da súmula
2. Um campeonato	E5 a E8	Entidades, inscrições, tabela, classificação e estatísticas
3. Piloto	E9 a E12	Páginas públicas, conta do atleta, retificação e torneio real

2. E0 — Fundação

A fundação termina quando uma rota de saúde da API e uma página vazia do front estão no ar, publicadas automaticamente a cada push.

2.1 Repositório

Um monorepo com workspaces separa os aplicativos dos pacotes de esporte. O módulo de esporte é TypeScript puro, usado pela API (validação oficial) e pelo front (resposta imediata na tela de mesa).

```
neronu/
  apps/
    api/                NestJS, Prisma, rotas REST
    web/                React PWA (mesa, painel, páginas públicas)
  packages/
    sport-core/         contrato SportModule e tipos comuns
    sport-basketball/   eventos, regras, estatísticas e layout do PDF
```

2.2 Ambientes

Componente	Serviço	Configuração inicial
API	Render	Deploy a partir da branch principal; variáveis de ambiente para banco e storage
Front	Vercel	Deploy a partir da branch principal; prévia por pull request

Componente	Serviço	Configuração inicial
Banco	Supabase (PostgreSQL)	Um projeto de desenvolvimento e um de produção; migrações via Prisma
PDFs	Supabase Storage	Um bucket privado para súmulas

2.3 Tarefas

- [ ] Criar o monorepo com workspaces, TypeScript, lint e formatação
- [ ] Criar `apps/api` com NestJS, Prisma e rota `GET /health`
- [ ] Criar `apps/web` com React, roteamento e manifesto de PWA
- [ ] Criar `packages/sport-core` e `packages/sport-basketball` vazios, já importados pelos dois apps
- [ ] Criar os projetos no Supabase e aplicar a primeira migração
- [ ] Configurar deploy no Render e na Vercel
- [ ] Configurar integração contínua: lint, testes e build a cada push

3. Marco 1 — Um jogo

O Marco 1 termina quando um amistoso ou treino real é anotado do início ao PDF da súmula, sem recorrer ao papel. Ainda não há login: o acesso é protegido por uma chave fixa de ambiente.

E1 — Módulo de basquete

Toda a regra do esporte fica pronta e testada antes de existir tela ou banco.

- [ ] Definir em `sport-core` o contrato `SportModule` e os tipos `MatchEvent`, `MatchState`, `RuleSet`
- [ ] Declarar `rulesSchema` e `defaultRules` do basquete (seção 8.2 do SDD)
- [ ] Implementar `reduce`: placar, faltas por atleta, faltas coletivas por período, tempos técnicos, exclusões
- [ ] Implementar `validate`: ordem dos períodos, atleta excluído, limite de tempos, prorrogação só com empate
- [ ] Tratar o evento `void` no `reduce`, recalculando o estado sem o evento anulado
- [ ] Implementar `playerStats`

- [ ] Escrever testes com sequências de eventos de jogos completos
- [ ] Conferir o contrato contra vôlei e futsal, no papel, para evitar suposições de basquete no núcleo

Pronto quando: um jogo inteiro, descrito como lista de eventos, produz placar e estatísticas corretos nos testes.

E2 — API de partida e eventos

- [ ] Modelar no Prisma: `Team`, `Athlete`, `Match`, `MatchRoster`, `MatchEvent`, `MatchSheet`
- [ ] Criar rotas de cadastro rápido de equipe e atleta
- [ ] Criar rota de partida avulsa, com as regras copiadas do padrão do módulo
- [ ] Criar rota de relacionados e titulares
- [ ] Criar `POST /matches/:id/events` com sequência por partida, chave de idempotência e validação pelo módulo
- [ ] Criar `GET /matches/:id/state`, que devolve eventos e estado calculado
- [ ] Proteger as rotas de escrita com a chave fixa de ambiente

Pronto quando: uma partida completa pode ser registrada só com chamadas à API, e reenviar o mesmo evento não o duplica.

E3 — Tela de mesa

É a etapa mais importante do projeto: decide se o produto é adotado.

- [ ] Tela de preparação: escolher equipes, relacionados e titulares
- [ ] Tela de jogo para celular: placar, período, faltas coletivas e duas listas de atletas
- [ ] Registrar cesta (1, 2, 3) e falta em até dois toques: atleta, depois ação
- [ ] Registrar tempo técnico e início e fim de período
- [ ] Aplicar o evento localmente com o `reduce` do módulo e enviar à API em seguida
- [ ] Manter fila local de eventos pendentes, com reenvio automático após queda de conexão
- [ ] Lista dos últimos eventos com botão de anular
- [ ] Sinalizar atleta excluído e equipe em bônus de faltas

Pronto quando: uma pessoa anota um jogo gravado em vídeo, em tempo real, sem perder lances.

E4 — Fechamento e PDF

- [ ] Tela de conferência: resumo do jogo antes de fechar
- [ ] `POST /matches/:id/close`: grava o hash dos eventos em `MatchSheet` e bloqueia novos eventos
- [ ] Criar em `sport-basketball` o layout da súmula com `@react-pdf/renderer`
- [ ] Gerar o PDF no servidor no fechamento e salvar no Supabase Storage
- [ ] Rota de download do PDF
- [ ] Anotar um amistoso ou treino real e corrigir o que atrapalhar na mesa

Pronto quando: o critério do Marco 1 é atingido.

4. Marco 2 — Um campeonato

O Marco 2 termina quando um campeonato simulado, com duas entidades, vai da criação à classificação final com estatísticas.

E5 — Login e entidades

- [ ] Modelar `User`, `Organization`, `Membership`
- [ ] Cadastro, login com JWT e verificação de e-mail
- [ ] Criar entidade e convidar membros com papel de administrador ou mesário
- [ ] Guardas de autorização por papel e por entidade em todas as rotas de escrita
- [ ] Substituir a chave fixa do Marco 1 pela autenticação real
- [ ] Telas de login, cadastro e painel inicial da entidade

Pronto quando: um membro de uma entidade não consegue ler dados privados nem escrever em outra.

E6 — Regras e cadastros

- [ ] Modelar `OrganizationSportRules`
- [ ] Tela de regras do módulo, gerada a partir do `rulesSchema`, mostrando todas as regras configuráveis
- [ ] Ligar `Athlete` e `Team` à entidade; e-mail e RGA opcionais, e-mail único por entidade
- [ ] Telas de atletas e equipes no painel da entidade

Pronto quando: a entidade altera uma regra padrão do basquete e uma nova partida já nasce com ela.

E7 — Campeonato e inscrições

- [ ] Modelar `Championship`, `ChampionshipInvite`, `Registration`, `RosterEntry`
- [ ] Criar campeonato copiando as regras padrão da organizadora; tela para editá-las no campeonato
- [ ] Gerar link de convite e fluxo de aceite pela entidade convidada
- [ ] Inscrição de equipe e montagem de elenco com números de camisa
- [ ] Aprovação ou recusa da inscrição pela organizadora
- [ ] Trava de elenco na data definida
- [ ] Equipe convidada: a organizadora cadastra equipe e atletas em nome de entidade sem conta (`isGuest`)

Pronto quando: duas entidades e uma equipe convidada estão inscritas e aprovadas no mesmo campeonato.

E8 — Tabela, classificação e estatísticas

- [ ] Modelar `Phase` e `Group`; ligar `Match` às inscrições e à fase
- [ ] Gerar grupos e tabela de jogos; gerar chave de mata-mata
- [ ] Implementar `standings` no módulo, com pontos por resultado e critérios de desempate
- [ ] Avançar equipes na chave a partir das súmulas fechadas
- [ ] Gravar `PlayerMatchStats` no fechamento da súmula
- [ ] Implementar `leaderboards` do basquete: cestinha por total de pontos, cestas de 3
- [ ] Telas de tabela, classificação, chave e rankings no painel
- [ ] Rodar o campeonato simulado completo

Pronto quando: o critério do Marco 2 é atingido.

5. Marco 3 — Piloto

O Marco 3 termina quando um torneio real da atlética é conduzido no Neronu, com resultados e estatísticas publicados.

E9 — Páginas públicas

- [ ] Rotas públicas de leitura, sem login, que devolvem só nome, número, equipe e estatísticas

- [ ] Página do campeonato: tabela, resultados, classificação e chave
- [ ] Destaques na página do campeonato: campeão e cestinha
- [ ] Página do atleta: estatísticas do campeonato atual e histórico por esporte quando houver
- [ ] Campo `publicProfile` e ocultação do perfil a pedido, feita pela entidade
- [ ] Declaração da entidade, no cadastro, de que tem autorização do atleta para publicar os dados

Pronto quando: um visitante sem conta acompanha o campeonato inteiro e nenhuma rota pública expõe e-mail ou RGA.

E10 — Identidade do atleta

- [ ] Ao verificar o e-mail ou fazer login, ligar a conta a todos os `Athlete` com o mesmo e-mail
- [ ] Área do atleta, somente leitura: campeonatos atuais, passados e futuros
- [ ] Área do atleta: estatísticas agrupadas por esporte, com colunas do `statsSchema`
- [ ] Transferência de equipe convidada para a entidade de destino
- [ ] Na transferência, unificar atletas pelo e-mail; sem e-mail, a entidade escolhe entre unificar ou manter
- [ ] Levar súmulas e estatísticas já registradas para o cadastro unificado

Pronto quando: um atleta cadastrado por duas entidades vê, em uma conta, os campeonatos das duas.

E11 — Retificação

- [ ] Rota de retificação de súmula fechada, restrita à organizadora, com motivo obrigatório
- [ ] Criar nova revisão em `MatchSheet`, com novo hash e novo PDF
- [ ] Recalcular `PlayerMatchStats`, classificação e chave após a retificação
- [ ] Exibir o histórico de revisões da súmula no painel

Pronto quando: corrigir uma cesta em jogo fechado atualiza PDF, classificação e ranking, e a versão anterior continua consultável.

E12 — Torneio piloto

- [ ] Combinar com a atlética o torneio e configurar as regras dela na tela do módulo
- [ ] Treinar os mesários em um jogo de teste

- [ ] Reativar o projeto do Supabase e conferir a API antes de cada rodada
- [ ] Acompanhar a primeira rodada presencialmente e registrar problemas
- [ ] Corrigir o que surgir entre as rodadas
- [ ] Fechar o torneio e recolher a avaliação da atlética

Pronto quando: o critério do Marco 3 é atingido.

6. API REST

A API segue recursos REST com JSON; a tabela lista as rotas principais na etapa em que cada uma é criada. Rotas de listagem e leitura simples dos mesmos recursos não estão repetidas.

Etapa	Método e rota	Função
E2	`POST /teams`	Cria equipe
E2	`POST /athletes`	Cria atleta
E2	`POST /matches`	Cria partida
E2	`PUT /matches/:id/roster`	Define relacionados e titulares
E2	`POST /matches/:id/events`	Registra evento (cabeçalho `Idempotency-Key`)
E2	`GET /matches/:id/state`	Eventos e estado calculado
E4	`POST /matches/:id/close`	Fecha a súmula
E4	`GET /matches/:id/sheet`	Baixa o PDF da súmula
E5	`POST /auth/register`	Cadastro de usuário
E5	`POST /auth/login`	Login
E5	`POST /auth/verify-email`	Confirma o e-mail
E5	`POST /organizations`	Cria entidade
E5	`POST /organizations/:id/members`	Adiciona membro com papel
E6	`GET` e `PUT /organizations/:id/sports/:sportKey/rules`	Lê e grava as regras padrão do esporte

Etapa	Método e rota	Função
E7	`POST /championships`	Cria campeonato com cópia das regras
E7	`PATCH /championships/:id`	Ajusta regras, datas e trava de elenco
E7	`POST /championships/:id/invites`	Gera link de convite
E7	`POST /invites/:token/accept`	Entidade aceita o convite
E7	`POST /championships/:id/registrations`	Inscreve equipe
E7	`PATCH /registrations/:id`	Aprova ou recusa inscrição
E7	`PUT /registrations/:id/roster`	Define o elenco
E7	`POST /championships/:id/guest-teams`	Cadastra equipe convidada
E8	`POST /championships/:id/schedule`	Gera grupos, tabela e chave
E8	`GET /championships/:id/standings`	Classificação
E8	`GET /championships/:id/leaderboards`	Rankings do campeonato
E9	`GET /public/championships/:slug`	Página pública do campeonato
E9	`GET /public/athletes/:id`	Página pública do atleta
E9	`PATCH /athletes/:id`	Edita cadastro, inclusive ocultação do perfil
E10	`GET /me/championships`	Campeonatos do atleta logado
E10	`GET /me/stats`	Estatísticas do atleta por esporte
E10	`POST /teams/:id/transfer`	Transfere equipe convidada
E11	`POST /matches/:id/rectifications`	Retifica súmula fechada

Convenções.

- Erros de regra do esporte voltam com status 422 e um código do módulo (por exemplo, `player_fouled_out`).
- Evento repetido com a mesma chave de idempotência devolve o evento já gravado, sem erro.

- Rotas `/public` nunca devolvem e-mail ou RGA.

7. Testes e qualidade

O esforço de teste se concentra no módulo de esporte e na API de eventos, onde um erro altera o resultado de um jogo.

Camada	O que testar	Como
Módulo de esporte	`validate`, `reduce`, `playerStats`, `standings`	Testes unitários com sequências de eventos; sem banco
API de eventos	Sequência, idempotência, bloqueio após fechamento	Testes de integração com banco de teste
Autorização	Acesso cruzado entre entidades e papéis	Testes de integração por rota de escrita
Rotas públicas	Ausência de e-mail e RGA nas respostas	Teste automatizado sobre o formato da resposta
Tela de mesa	Rapidez e fila de reenvio	Teste manual com jogo gravado; simulação de queda de conexão
PDF	Layout da súmula	Conferência visual contra uma súmula em papel

Definição de pronto de uma tarefa.

- Código na branch principal, com lint e testes passando.
- Publicado nos ambientes do Render e da Vercel.
- Migração de banco aplicada, quando houver.
- Item marcado neste plano.

8. Pontos de atenção

Quatro pontos podem atrasar ou comprometer a execução; cada um tem uma ação prevista em uma etapa.

Ponto	Onde aparece	Ação
Tela de mesa lenta ou confusa	E3	Testar com jogo gravado antes do jogo real; só avançar ao Marco 2 depois do critério do Marco 1
Banco do Supabase pausado por inatividade no plano gratuito	E12 e intervalos entre torneios	Reativar e testar antes de cada rodada
API indisponível ou lenta no início do jogo	E3 e E12	Abrir a tela de mesa alguns minutos antes; a fila local segura os eventos
Escopo crescendo antes do piloto	Todas	Novas ideias vão para a lista pós-MVP do SDD, não para a etapa em curso

Ordem de trabalho. As etapas são feitas em sequência, de E0 a E12. Dentro de uma etapa, a ordem das tarefas é a da lista. A única antecipação recomendada é conseguir cedo, com a atlética, uma súmula de basquete em papel preenchida, que serve de referência para E1, E3 e E4.